Computers and Fluids 245 (2022) 105583

Contents lists available at ScienceDirect

Computers and Fluids

journal homepage: www.elsevier.com/locate/compfluid

RPINNs: Rectified-physics informed neural networks for solving stationary partial differential equations

Pai Peng [a], Jiangong Pan [b], Hui Xu [c,*], Xinlong Feng [a]

[a] College of Mathematics and System Sciences, Xinjiang University, Urumqi 830046, PR China
[b] Department of Mathematical Sciences, Tsinghua University, Beijing 100084, PR China
[c] School of Aeronautics and Astronautics, Shanghai Jiao Tong University, Shanghai 200240, PR China

ARTICLE INFO

Keywords:
Rectified physics-informed neural networks
Multigrid
Neural tangent kernel
Dynamic weight strategy
High precision

ABSTRACT

Due to the development of high performance computing, deep learning algorithm has made a significant progress in many fields such as computational mathematics. The physics-informed neural networks have put forward a innovative idea for tackling a broad range of forward and inverse problems of partial differential equations. Motivated by the philosophy of physics-informed neural network and the multigrid method, we introduce the gradient information of numerical solution of physics-informed neural network into the new neural networks and propose the rectified-physics informed neural network for solving stationary partial differential equations. And for solving multi-objective optimization of neural networks, the dynamic weight strategy is adopted to balance numerical difference among terms in the loss function, and effectively alleviate the gradient ill-conditioned phenomenon. Finally, we perform a series of numerical experiments to demonstrate effectiveness of the RPINNs method which is combined with the dynamic weight strategy to improve calculation accuracy.

1. Introduction

In recent years, continuous growth of computer computing power has allowed rapid development of machine learning [1,2], which has highlighted quite effective applications in various disciplines, such as face recognition, data mining, and application of intelligent machines. The field of deep learning have brought an important impact on the way we handle multitasking and heavy tasks [3–6]. As a kind of nonlinear approximator, neural network has a powerful approximation ability. Although the nonlinear approximation has higher performance, the nonlinear characteristic brings additional difficulties. It is still a challenging task to achieve the best approximation. So far, how to give a priori knowledge to the neural network, a powerful function approximator, has become a focus of researches [7]. Some scholars are dedicated to designing specialized neural network to implicitly embed given prior knowledge into the neural network structure, such as using convolutional neural networks to process computer vision [3]. Darbon et al. proposed two neural network architectures which naturally encode the physics to represent viscosity solutions of Hamilton Jacobi partial differential equations without using grids or numerical approximations [8,9]. Some researches aimed to give prior knowledge to neural network by appropriately penalizing loss function of neural network. Among them, the most representative neural network is the so-called physics-informed neural network which is used to tackle the forward and inverse problems of partial differential equations.

In 2019, Raissi, et al. proposed a physics-informed neural network to solve forward and inverse problems of partial differential equations [10–12]. Subsequently, a series of research results are explored based on different equations and applications [13]. In the application field, Jin et al. solved the problem of laminar flow and channel turbulence by establishing NSF-nets in the form of velocity–pressure and velocity–vorticity [14]. Wessels et al. proposed the Neural Particle Method to solve the incompressible free surface flow subject to the Euler equations [15]. Kissas et al. established a model for cardiovascular flow, which could predict the flood flow in the cardiovascular system [16]. In addition, some researchers incorporated the traditional method of solving partial differential equations into PINNs, for example, based on the idea of solving the partial differential equations with finite volume, Japtap et al. proposed DPINNs by adding interface constraints as regularization factors to the loss function [17]. E.Kharazmi et al. proposed VPINNs to solve partial differential equations based on the Petrov–Galerkin variational principle [18]. In addition, the theoretical exploration of physics-informed neural networks is also concerned by researchers, such as the estimation of generalization error of neural networks and the optimization of activation function.

* Corresponding author.
E-mail addresses: penguinmath@163.com (P. Peng), mathpjg@sina.com (J. Pan), dr.hxu@sjtu.edu.cn (H. Xu), fxlmath@xju.edu.cn (X. Feng).

https://doi.org/10.1016/j.compfluid.2022.105583
Received 29 December 2021; Received in revised form 5 June 2022; Accepted 28 June 2022
Available online 30 June 2022

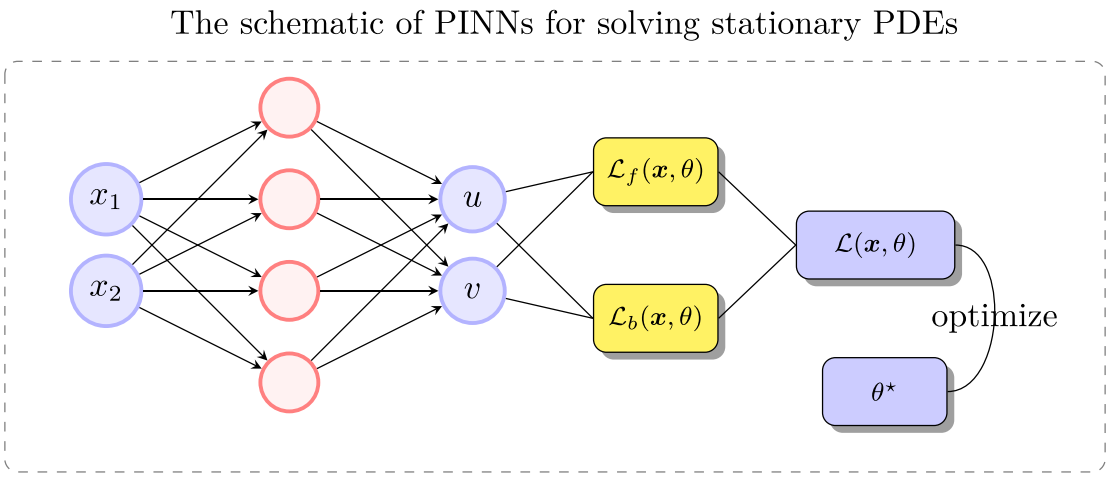


Fig. 1. Schematic diagram of the structure of the physics informed neural network for solving stationary partial differential equations.

Mishra et al. explored the data assimilation and unique continuation problems and provided a strict estimate of the generalization error of PINNs approximating them [19]. And De Ryck et al. certificated the rigorous bounds of the errors produced by PINNs in approximating the solutions of a large class of linear parabolic partial differential equations including the Kolmogorov equations [20]. By applying the maximum principle and Schauder approach, Shin et al. proved that the sequence of minimizers generated by PINNs strongly converges to the PDE solutions in C_0[21]. Lanthaler et al. analysed the estimates on the resulting approximation and generalization error of DeepOnets and evaluated the upper and lower bounds of the total errors [22].

Based on the philosophy of solving partial differential equations on multigrid, we here propose a new scheme, which is called '*Rectified physics informed neural networks*' (RPINNs). As we all know, the multigrid algorithm was originally originated from the iterative solution of the grid system, which constrains the remaining part of the fine-grid equation to be solved on the coarse grid, and then extends the obtained solution to the fine grid. RPINNs is designed from this points, it transfers the gradient of the numerical solution obtained by PINNs to the new neural networks, and establishes the loss function for it to obtain the correction of numerical solutions. Meanwhile, in order to overcome the gradient ill-conditioned problem that appears in training process, we adopt two different dynamic weight strategies in optimization process of RPINNs to balance numerical discrepancies among terms in loss function.

This paper are organized as follow. In Section 2, we give a brief introduction of the physics-informed neural networks. To alleviate the ill-conditioned phenomenon of gradients, we introduce dynamic weight strategies based on gradient information and analysis of neural tangent kernel (NTK) of PINNs [23] in Section 3. In Section 4, we introduce in detail the proposed RPINNs. In Section 5, by a series of numerical experiments, we demonstrate that RPINNs combined with dynamic weight strategies can improve calculation accuracy. Finally, we summarize the work in Section 6.

2. Physics-informed neural networks

In this section, we introduce the process of solving stationary partial differential equations by PINNs. We consider the following stationary partial differential equations in operator forms on the bounded domain $\Omega \subset R^d$:

$$\begin{aligned} &\mathcal{N}_x[\boldsymbol{u}] = f(\boldsymbol{x}), \quad \boldsymbol{x} \in \Omega, \\ &\mathcal{B}[\boldsymbol{u}] = g(\boldsymbol{x}), \quad \boldsymbol{x} \in \partial\Omega, \end{aligned} \tag{1}$$

where $\mathcal{N}_x$ is a differential operator, $\mathcal{B}$ is the boundary conditions which could be Dirichlet, Neumann or Robin. $\boldsymbol{u}(\boldsymbol{x})$ represents unknown that satisfies the operator equation, and $g(\boldsymbol{x})$ is a function on the boundary $\partial\Omega$. As we know, the neural network establishes a mapping relationship from input to output. In mathematics, we can express neural network as a complex composite function:

$$f(\boldsymbol{x},\theta)_L = W_L(\cdots \sigma(W_1\sigma(W_0\boldsymbol{x} + b_0) + b_1)) + b_L, \tag{2}$$

where $\boldsymbol{x} \in R^d$ denotes input vector, $\theta = \{W_i, b_i\}_{i=0}^{L}$ represents the set of all weight matrices and bias vectors, which are initialized using the Xavier scheme. $\sigma(\cdot)$ denotes the activation function of neural network which we usually adopt the hyperbolic tangent function $\sigma(\cdot) = (e^x - e^{-x})/(e^x + e^{-x})$.

In solving partial differential equations with the PINNs proposed by Raissi et al. the neural network $f(\boldsymbol{x},\theta)_L$ can be regarded as a potential solution of Eq. (1), which have the residual form $\boldsymbol{r}_\theta(\boldsymbol{x})$:

$$\boldsymbol{r}_\theta(\boldsymbol{x}) = \mathcal{N}_x[f(\boldsymbol{x},\theta)_L] - f(\boldsymbol{x}), \tag{3}$$

with the constraint

$$f(\boldsymbol{x},\theta)_L(\boldsymbol{x}) = g(\boldsymbol{x}), \tag{4}$$

on the boundary. The gradient of the nonlinear function $f(\boldsymbol{x},\theta)_L(\boldsymbol{x})$ with respect to the space vector $\boldsymbol{x}$ can be obtained by the automatic differentiation mechanism which is a method of calculating the derivative of a polynomial function [24]. It can be used to quickly calculate gradient of a function by using the chain rule and other differentiation principles, as well as operator overloading and code conversion technology. Traditionally, gradient of neural network can also be obtained by finite difference method.

If the nonlinear function $f(\boldsymbol{x},\theta)_L$ satisfies partial differential equations, then the residual $\mathbf{r}_\theta(\boldsymbol{x}) = 0$. According to residual and boundary conditions of the equation, the loss function of the neural network can be established to satisfy:

$$\mathcal{L}(\boldsymbol{x},\theta) = \lambda_f\mathcal{L}_f(\boldsymbol{x},\theta) + \lambda_b\mathcal{L}_b(\boldsymbol{x},\theta), \tag{5}$$

where

$$\mathcal{L}_f(\boldsymbol{x},\theta) = \frac{1}{N_f}\sum_{i=1}^{N_f} |\boldsymbol{r}_\theta(\boldsymbol{x}_f^i)|^2,$$

$$\mathcal{L}_b(\boldsymbol{x},\theta) = \frac{1}{N_b}\sum_{i=1}^{N_b} |f(\boldsymbol{x},\theta)_L(\boldsymbol{x}_b^i) - g(\boldsymbol{x}_b^i)|^2,$$

λ_f and λ_b respectively represent weight coefficients of the loss function $\mathcal{L}_f(\boldsymbol{x},\theta)$ and $\mathcal{L}_b(\boldsymbol{x},\theta)$. N_f and N_b denote size of training data $\{\boldsymbol{x}_f^i, f(\boldsymbol{x}_f^i)\}_{i=1}^{N_f}$ and $\{\boldsymbol{x}_b^i, g(\boldsymbol{x}_b^i)\}_{i=1}^{N_b}$ respectively, which could be obtained by LHS method (a method of approximately random sampling from a multivariate parameter distribution and often used in computer experiments [25]) or uniformly split method.

In order to make the loss function $\mathcal{L}(\boldsymbol{x},\theta)$ tend to 0, a series of optimization algorithms, such as Adam [26] and L-BFGS-B [27], can be used to optimize the parameters of the neural network. Finally, the numerical solution of the PINNs method is expressed as: $u^*(\boldsymbol{x}) = f(\boldsymbol{x},\theta^\star)_L$. The process of PINNs to solve stationary partial differential equations is shown in Fig. 1.

3. Dynamic weight strategy

The selection of weight coefficients λ_f and λ_b in the loss $\mathcal{L}(\boldsymbol{x},\theta)$ affects the training effect of neural network. However, choosing appropriate weight coefficients for neural network is generally very difficult. In this section, in order to select the appropriate weight coefficients, we introduce two dynamic weight strategies based on neural network gradient and neural tangent kernel respectively. Meanwhile, for convenience, when the weight coefficients $\lambda_f = 1$ and $\lambda_b = 1$, the dynamic weight strategy is defined as (D_0).

3.1. Dynamic weight strategy based on neural network gradient

In order to avoid unnecessary consumption of time due to repeated trial and error in adjusting the weight coefficients, we follow the original work of Wang S and adopt the dynamic weight strategy based on neural network gradient, which the main idea is to dynamically adjust the weight coefficient λ_b by utilizing the gradient in back-propagated during neural network training.

In order to sufficiently and reasonably explain the rationality of adopting dynamic weights, we consider the following continuous time gradient flow

$$\frac{\mathrm{d}\theta}{\mathrm{d}t} = -\nabla\mathcal{L}(\boldsymbol{x},\theta). \tag{6}$$

From the original work of [7], the author analysed the reasons for failure of PINNs related to stiffness in the gradient flow system, which leads to instability of the gradient value between the various items in the loss function $\mathcal{L}(\boldsymbol{x},\theta)$ in back-propagated. In order to alleviate gradient pathologies [28], we here adopt a dynamic weight strategy (D_1) which based on neural network gradient to balance terms in the loss function $\mathcal{L}(\boldsymbol{x},\theta)$ [7]. As described in Algorithm 1, the weight coefficient λ_b is determined by gradient statistics. At the same time, in order to reduce to the calculation cost, we could set a hyper-parameter to determine the iteration step size.

3.2. Dynamic weight strategy based on neural tangent kernel theory

Following the original work of Wang et al. [29], from the perspective of the neural tangent kernel (NTK) of PINNs, the authors proved that, under the assumption of LeCun initialization for network parameters, the NTK of PINNs converges to a deterministic kernel and maintain constant during training with a small learning rate. And the authors analysed that PINNs suffer from spectral bias [30] and a significant discrepancy of convergence rate in different terms of loss function in the gradient flow system, and proposed a novel adaptive training strategy (D_2) to strengthen predictive accuracy of PINNs. As described in Algorithm 2, the determination of the weight coefficients λ_f and λ_b depends on the eigenvalues of the neural tangent kernel corresponding to different losses. And the definition and formula of neural network tangent kernel matrix are shown in Remark 3.1 (refer to the Ref. [29]).

Remark 3.1. let $J_r(t)$ and $J_b(t)$ is the Jacobian matrices of $\mathcal{L}_f(\boldsymbol{x},\theta(t))$ and $\mathcal{L}_b(\boldsymbol{x},\theta(t))$ with respect to parameters θ at time t in the continuous time gradient flow system, respectively. We could deduce that

$$\mathbf{K}_r(t) = \mathbf{J}_r(t)\mathbf{J}_r(t)^T, \mathbf{K}_b(t) = \mathbf{J}_b(t)\mathbf{J}_b(t)^T,$$

$$\mathbf{K}(t) = \begin{pmatrix} \mathbf{J}_r(t) \\ \mathbf{J}_b(t) \end{pmatrix} \begin{pmatrix} \mathbf{J}_r(t), \mathbf{J}_b(t) \end{pmatrix}.$$

Algorithm 1 Learning rate annealing for physics informed neural networks [7] (D_1)

Consider the physics-informed neural networks $f(\boldsymbol{x},\theta)_L$ with loss function

$$\mathcal{L}(\boldsymbol{x},\theta) = \lambda_f\mathcal{L}_f(\boldsymbol{x},\theta) + \lambda_b\mathcal{L}_b(\boldsymbol{x},\theta), \tag{7}$$

where λ_f and λ_b are parameters used to balance the interplay between the various terms of the loss function. When we adopt the gradient descent algorithm to optimize the neural network, the update formula of parameter θ is:

$$\theta_{k+1} = \theta_k - \eta\lambda_f\nabla_\theta\mathcal{L}_f - \eta\lambda_b\nabla_\theta\mathcal{L}_b, \tag{8}$$

η denotes the learning rate of the neural network.

1. Consider the iterative change of λ_b in the optimization process, and calculate the intermediate variable:

$$\hat{\lambda_b} = \frac{\max\{|\nabla_\theta\mathcal{L}_f(\theta_k)|\}}{\overline{|\nabla_\theta\mathcal{L}_b(\theta_k)|}}, \tag{9}$$

where $\max\{|\nabla_\theta\mathcal{L}_f(\theta_k)|\}$ and $\overline{|\nabla_\theta\mathcal{L}_b(\theta_k)|}$ denotes the max of $|\nabla_\theta\mathcal{L}_f(\theta_k)|$ and the mean of $|\nabla_\theta\mathcal{L}_b(\theta_k)|$ with respect to parameters θ, respectively.

2. According to the intermediate variable $\hat{\lambda_b}$ and the weight value of $\mathcal{L}_b(\boldsymbol{x},\theta)$ during the kth iteration $\lambda_b^{(k)}$, compute the weight in the k+1th iteration $\lambda_b^{(k+1)}$:

$$\lambda_b^{(k+1)} = (1-\mu)\lambda_b^{(k)} + \mu\hat{\lambda_b}, \tag{10}$$

The commended hyper-parameters values are: $\lambda_f = 1$, $\eta = 10^{-3}$, $\mu = 0.9$.

the ijth component of $\mathbf{K}_r(t)$ is

$$(\mathbf{K}_r)_{ij}(t) = \left\langle \frac{\mathrm{d}\mathcal{L}_f(\boldsymbol{x}_f^i,\theta(t))}{\mathrm{d}\theta}, \frac{\mathrm{d}\mathcal{L}_f(\boldsymbol{x}_f^j,\theta(t))}{\mathrm{d}\theta} \right\rangle.$$

4. Rectified physics informed neural networks (RPINNs)

We now propose the rectified physics-informed neural network to solve partial differential equations based on the idea of the multi-grid method. The multi-grid method uses grids of different scales to eliminate the error components of different wavelengths, and eliminates various error components layer by layer. According to the process of solving partial differential equations on multigrid, we could roughly calculate the numerical solution of PINNs, and then use another neural network to rectify it through the gradient information.

We observe that if the numerical solution obtained by solving the PINNs is taken into Eq. (1), we could infer that the formula satisfies

$$\begin{aligned} &\mathcal{N}_x[\boldsymbol{u}^*] \approx f(\boldsymbol{x}),\ \boldsymbol{x}\in\Omega, \\ &\mathcal{B}[\boldsymbol{u}^*] \approx g(\boldsymbol{x}),\ \boldsymbol{x}\in\partial\Omega, \end{aligned} \tag{17}$$

It is clear that there exists an error between numerical solution $\boldsymbol{u}^*$ and true solution $\boldsymbol{u}$ which is denoted by $\boldsymbol{\varepsilon} = \boldsymbol{u} - \boldsymbol{u}^*(\boldsymbol{x})$. In the next step, we construct a new neural network for $\boldsymbol{\varepsilon}(\boldsymbol{x},\theta)$ to satisfy the physics imposed by the operator equation and boundary conditions.

$$\begin{aligned} &\mathcal{N}_x[\boldsymbol{u}^* + \boldsymbol{\varepsilon}(\boldsymbol{x},\theta)_L] = f(\boldsymbol{x}), \quad \boldsymbol{x}\in\Omega, \\ &\mathcal{B}[\boldsymbol{u}^* + \boldsymbol{\varepsilon}(\boldsymbol{x},\theta)_L] = g(\boldsymbol{x}), \quad \boldsymbol{x}\in\partial\Omega. \end{aligned} \tag{18}$$

Meanwhile, we define the residual of the equation as

$$\boldsymbol{r}_\theta^*(\mathbf{x}) = \mathcal{N}_x[\boldsymbol{u}^* + \boldsymbol{\varepsilon}(\boldsymbol{x},\theta)_L] - f(\boldsymbol{x}) \tag{19}$$

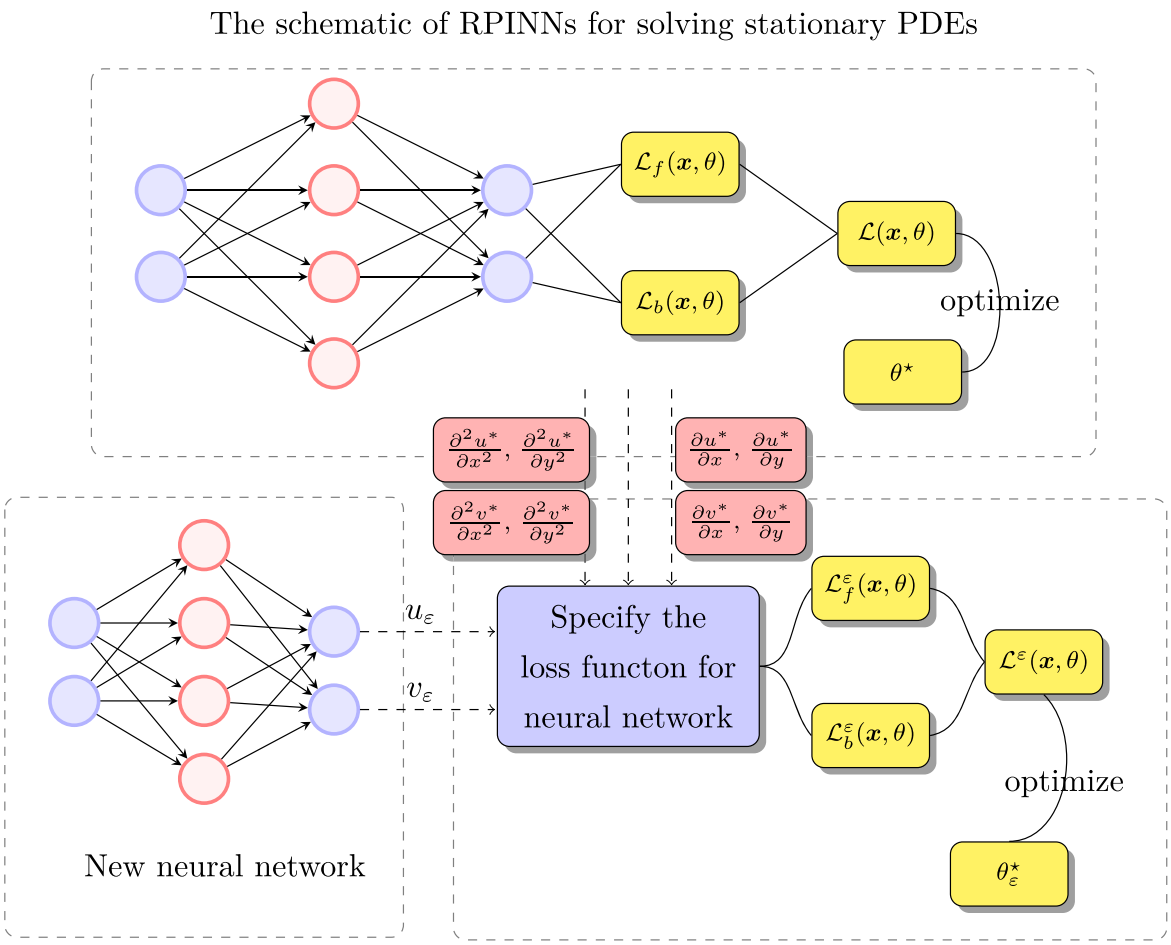


Fig. 2. Schematic diagram of the structure of the rectified physics informed neural network for solving stationary partial differential equations.

To measure discrepancy between the numerical solution and the true solution, we consider the loss function defined as the weighted summation of the L_2 norm of error $\boldsymbol{\varepsilon}$:

$$\mathcal{L}^\varepsilon(\boldsymbol{x},\theta) = \lambda_f^\varepsilon \mathcal{L}_f^\varepsilon(\boldsymbol{x},\theta) + \lambda_b^\varepsilon \mathcal{L}_b^\varepsilon(\boldsymbol{x},\theta), \tag{20}$$

where

$$\mathcal{L}_f^\varepsilon(\boldsymbol{x},\theta) = \frac{1}{N_f^\varepsilon}\sum_{i=1}^{N_f^\varepsilon} |r_\theta^*(\boldsymbol{x}_f^i)|^2, \tag{21}$$

$$\mathcal{L}_b^\varepsilon(\boldsymbol{x},\theta) = \frac{1}{N_b^\varepsilon}\sum_{i=1}^{N_b^\varepsilon} |\varepsilon(\boldsymbol{x},\theta)_L(\boldsymbol{x}_b^i) - \boldsymbol{u}^*(\boldsymbol{x}_b^i) - g(\boldsymbol{x}_b^i)|^2, \tag{22}$$

λ_f^ε and λ_b^ε respectively represent the weight coefficient between the loss function $\mathcal{L}_f^\varepsilon(\boldsymbol{x},\theta)$ and $\mathcal{L}_b^\varepsilon(\boldsymbol{x},\theta)$. N_f^ε and N_b^ε denote the size of the training data $\{\boldsymbol{x}_f^i, f(\boldsymbol{x}_f^i)\}_{i=1}^{N_f^\varepsilon}$ and $\{\boldsymbol{x}_b^i, g(\boldsymbol{x}_b^i)\}_{i=1}^{N_b^\varepsilon}$ respectively. $\boldsymbol{u}^*(\boldsymbol{x}_b^i)$ denotes the numerical solution obtained by PINNs in the boundary condition. In order to effectively improve the effectiveness of our method, the training set $\{\boldsymbol{x}_f^i, f(\boldsymbol{x}_f^i)\}_{i=1}^{N_f^\varepsilon}$ and the training set $\{\boldsymbol{x}_f^i, f(\boldsymbol{x}_f^i)\}_{i=1}^{N_f}$ of PINNs are independent of each other.

At the last step, in order to search for a good θ for the loss function $\mathcal{L}^\varepsilon(\boldsymbol{x},\theta)$, we adopt Adam and L-BFGS-B optimizers to minimize the loss function. In this way, we can easily obtain the correction value $\boldsymbol{\varepsilon}^* = \boldsymbol{\varepsilon}(\boldsymbol{x},\theta)_L$ for the numerical solution of PINNs. Finally, we can express the numerical solution obtained by the RPINNs as $\boldsymbol{u}_R = \boldsymbol{u}^*(x) + \boldsymbol{\varepsilon}(\boldsymbol{x},\theta)_L$. In Fig. 2, we show the flow chart of RPINNs for solving stationary partial differential equations. The key benefit of the proposed correction strategy is that we use different neural networks to solve the numerical results which the first network obtains the approximate solution of partial differential equations, and the other neural network rectify the approximate solution.

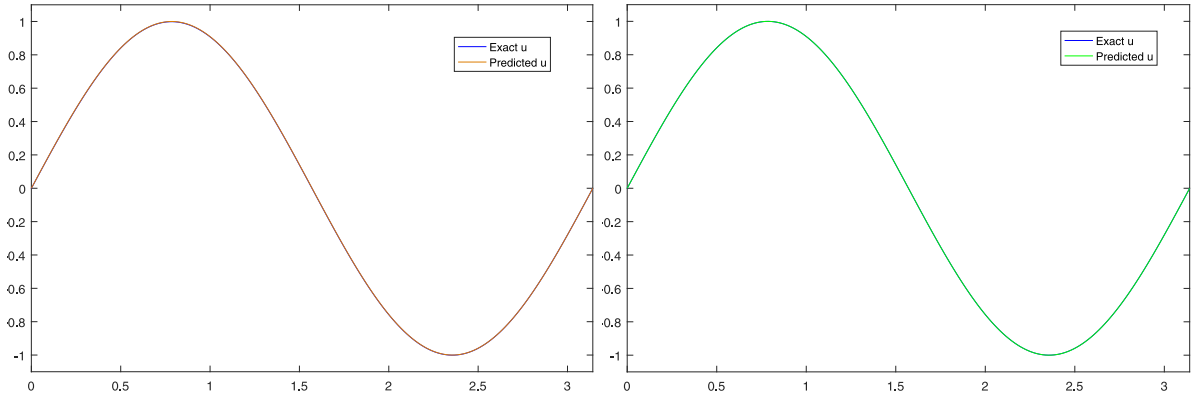
5. Numerical results

In this section we provide a series of numerical experiments to evaluate effectiveness of the proposed method in improving numerical accuracy. We compare the effectiveness of RPINNs combined with different dynamic weight strategies in improving the computational accuracy. And all dynamic weight strategies are described in Section 3. In all cases we employ the hyperbolic tangent activation function. Moreover, all networks are initialized using the Glorot method, the value of the parameter λ_f and λ_b are updated every 5 iterations of Adam. Meanwhile, the code based on the tensorflow framework and the reported time are measured on VivoBook ASUSLaptop X571GT VX60GT with an Intel Core i7-9750H CPU processor.

5.1. Automatic differentiation & finite difference

To evaluate the discrepancy between the automatic differentiation and the finite difference in calculating the gradient in the neural network, we consider the solution of the one-dimensional Poisson equation. The problem in one spatial dimension takes the form:

$$\begin{cases} -\Delta u(x) = f(x), & x \in \Omega, \\ u(x) = 0, & x = 0, \\ u(x) = 0, & x = \pi, \end{cases} \tag{23}$$



(a) Finite difference (b) Automatic differentiation

Fig. 3. Comparison of numerical solution results obtained by calculating gradients using finite difference and automatic differentiation.

Algorithm 2 Learning rate annealing for physics-informed neural networks [29] (D_2)

Consider the physics-informed neural networks $f(\boldsymbol{x},\theta)_L$ with loss function

$$\mathcal{L}(\boldsymbol{x},\theta) = \lambda_f \mathcal{L}_f(\boldsymbol{x},\theta) + \lambda_b \mathcal{L}_b(\boldsymbol{x},\theta), \tag{11}$$

where λ_f and λ_b are parameters used to balance the interplay between the various terms of the loss function. When we adopt the gradient descent algorithm to optimize the neural network, the update formula of parameter θ is:

$$\theta_{k+1} = \theta_k - \eta\lambda_f \nabla_\theta \mathcal{L}_f - \eta\lambda_b \nabla_\theta \mathcal{L}_b, \tag{12}$$

η denotes the learning rate of the neural network.

1. Compute parameters λ_f and λ_b as follow:

$$\hat{\lambda_f} = \frac{\sum_{i=1}^{N_f+N_b} \lambda_i(n)}{\sum_{i=1}^{N_f} \lambda_i^r(n)} = \frac{Tr(\mathbf{K}(n))}{Tr(\mathbf{K}_r(n))}, \tag{13}$$

$$\hat{\lambda_b} = \frac{\sum_{i=1}^{N_f+N_b} \lambda_i(n)}{\sum_{i=1}^{N_b} \lambda_i^b(n)} = \frac{Tr(\mathbf{K}(n))}{Tr(\mathbf{K}_b(n))}, \tag{14}$$

where $\lambda_i(n)$, $\lambda_i^r(n)$ and $\lambda_i^b(n)$ are eigenvalues of $\mathbf{K}(n)$, $\mathbf{K}_r(n)$ and $\mathbf{K}_b(n)$.

2. According to the intermediate variable $\hat{\lambda_b}$ and the weight value of $\mathcal{L}_b(\boldsymbol{x},\theta)$ during the kth iteration $\lambda_b^{(k)}$, compute the weight in the k+1th iteration $\lambda_b^{(k+1)}$:

$$\lambda_f^{(k+1)} = (1-\mu)\lambda_f^{(k)} + \mu\hat{\lambda_f}, \tag{15}$$

$$\lambda_b^{(k+1)} = (1-\mu)\lambda_b^{(k)} + \mu\hat{\lambda_b}, \tag{16}$$

The commended hyper-parameters values are: $\eta = 10^{-3}$, $\mu = 0.9$.

where $\Omega = [0,\pi]$, the function $f(x)$ is known, and the function $u(x)$ is the latent solution represented by a neural network. To evaluate the discrepancy, we will use a solution like:

$$u(x) = \sin(x). \tag{24}$$

and derive a forcing term $f(x)$ correspondingly.

Table 1
Comparison between using finite difference and automatic differentiation to solve equations using PINNs.

Operator	Optimization	Iterations	Network structure	Time (s)	Absolute error
FD	Adam	2000	[1, 20, 20, 1]	157.856	5.5245E−04
AD	Adam	2000	[1, 20, 20, 1]	13.996	2.1822E−05

The neural network $f(x,\theta)_L$ could be trained to approximate the latent solution $u(x)$ by establishing the follow loss:

$$\mathcal{L}(x,\theta) = \lambda_f \mathcal{L}_f(x,\theta) + \lambda_b \mathcal{L}_b(x,\theta), \tag{25}$$

If we use finite difference to calculate the gradient of the neural network $f(x;\theta)_L$ with respect to x, $\mathcal{L}_f(x,\theta)$ could be defined as

$$\mathcal{L}_f(x,\theta) = \sum_{i=1}^{n-1} |-\frac{u_{i+1}-2u_i+u_{i-1}}{h^2} - f(x_i)|^2, \tag{26}$$

while the loss function $\mathcal{L}_b(x,\theta)$ which satisfies the dirichlet boundary condition can be defined as follow:

$$\mathcal{L}_b(x,\theta) = |u_0 - a|^2 + |u_n - b|^2. \tag{27}$$

Without loss of generality, we consider an example in which $f(x,\theta)_L$ is a 4-layer deep neural network with 20 neurons per layers. The dimension of training data $\{x_f^i, f(x_f^i)\}_{i=1}^{N_f}$ is 511, which is obtained by uniformly dividing the region Ω. Meanwhile, to balance the discrepancy between the loss function $\mathcal{L}_f(x,\theta)$ and $\mathcal{L}_b(x,\theta)$, let us consider the hyper-parameters $\lambda_f = 1$ and $\lambda_b = 10$. And we train the neural network for 2000 stochastic gradient descent steps by minimizing the loss function using the Adam optimizer which the initial learning rate is set $\eta = 10^{-3}$. (See Table 1.)

As illustrated in Fig. 3, we compare the numerical solution obtained by the finite difference method and the automatic differentiation method to solve the gradient of the neural network with respect to x. Table 1 summarizes the results, in the case of the same neural network structure and the same stochastic gradient step, using the automatic differentiation mechanism to calculate the gradient information is more efficient in accuracy and time.

In addition, due to the finite difference calculates the gradient, the information flow propagates from the boundary to the internal nodes, if the training process at the boundary is bad, the neural network training will give rise to a problem. The result shows the significance of using a dynamic weight strategy to balance the loss from another perspective.

5.2. Convection diffusion reaction equation

To evaluate the ability of the RPINNs method combined with the dynamic weight strategy (D_1) to handle the linear problems, let us

Table 2
The numerical comparison of Multigrid, PINNs and RPINNs Methods for solving convection diffusion reaction equation.

Method	Multi-grid	PINNs (D_0)	PINNs (D_1)	RPINNs (D_0)	RPINNs (D_1)
Time (s)	109.32	830.56	901.82	897.78	971.14
Number of points	262144	13300	13300	13300	13300
Absolute error	2.0178E−03	1.0175E−03	8.1785E−04	3.8411E−04	1.1389E−04
Relative L_2 error	1.3352E−02	6.6352E−03	4.2769E−03	2.1830E−03	7.4697E−04

consider the convection diffusion reaction equation which play a significant role in describing many physical phenomena such as atmospheric pollution, river pollution phenomenon. The equation form is given as follows:

$$\begin{cases} -\alpha\Delta u(x,y)+\beta\nabla u(x,y)+u(x,y)=f(x,y) & \text{in } \Omega, \\ u(x,y)=g(x,y) & \text{on } \partial\Omega, \end{cases} \tag{28}$$

where α represents the diffusion term coefficient, $\beta=(2,3)$ represents the convection term coefficient. The functions f and g are considered to be known, the function u denotes the numerical solution that could be calculated through a neural network. In this problem, we defined the computational domain is $\Omega=[0,1]^2$. Correspondingly, we choose the boundary conditions satisfying $g=0$ for all $x\in\partial\Omega$. To assess the accuracy of our models we shall use a fabricated solution

$$u(x,y)=2\sin(x)(1-e^{\frac{-2(1-x)}{\alpha}})y^2(1-e^{\frac{y-1}{\alpha}}). \tag{29}$$

and correspondingly derive a forcing term f which satisfies the equation.

In our proposed model, a physics-informed neural network $f(\boldsymbol{x},\theta)_L$ can be trained to approximate the numerical solution by constructing the loss function:

$$\mathcal{L}(\boldsymbol{x},\theta)=\lambda_f\mathcal{L}_f(\boldsymbol{x},\theta)+\lambda_b\mathcal{L}_b(\boldsymbol{x},\theta), \tag{30}$$

where

$$\mathcal{L}_f(\boldsymbol{x},\theta)=\frac{1}{N_f}\sum_{i=1}^{N_f}|-\alpha\Delta u_i+\beta\nabla u_i+u_i-f_i|^2, \tag{31}$$

and

$$\mathcal{L}_b(\boldsymbol{x},\theta)=\frac{1}{N_b}\sum_{i=1}^{N_b}|u_i-g_i|^2, \tag{32}$$

Observe that if the parameter $\lambda_f=1$ and $\lambda_b=1$, the model degenerates to the original physics-informed neural network formulation of Raissi, et al. N_f, N_b respectively represent the dimensionality of the training data which satisfying the loss $\mathcal{L}_f(\boldsymbol{x},\theta)$ and $\mathcal{L}_b(\boldsymbol{x},\theta)$.

Without loss of generality, let us consider an forecast model in which $f(\mathbf{x};\theta)_L$ is a 4-layer neural network accompanied by 50 neurons per layer and a hyperbolic tangent activation function. We train the network for 3000 gradient descent steps to minimize the loss function by using the Adam optimizer which the initial learning rate is set $\eta=10^{-3}$. Meanwhile, in the process of training, the decay strategy is adopted for the change of learning rate. After 1000 iterations, the learning rate decays to 0.9 times of the original learning rate. Besides, we set the other parameters of this study as $N_f=8000$, $N_b=2000$. And we use the dynamic weight strategy D_1 to balance the discrepancies of each item in the loss function.

Notice that if we take the numerical solution $\hat{u}$ obtained by PINNs into Eq. (37), we can infer that

$$\begin{aligned} &-\alpha\Delta\hat{u}+\beta\nabla\hat{u}+\hat{u}\approx f && \text{in } \Omega, \\ &\hat{u}\approx g && \text{on } \partial\Omega, \end{aligned} \tag{33}$$

Since in Eq. (33), the rough numerical solution $\hat{u}$ does not exactly satisfy the equation, which is always an error to satisfy $\varepsilon=\hat{u}-u$. To measure ε, we construct a new fully connected neural network $\varepsilon(\boldsymbol{x},\theta)_L$ in which the loss function is defined as

$$\mathcal{L}^\varepsilon(\boldsymbol{x},\theta)=\lambda_f^\varepsilon\mathcal{L}_f^\varepsilon(\boldsymbol{x},\theta)+\lambda_b^\varepsilon\mathcal{L}_b^\varepsilon(\boldsymbol{x},\theta), \tag{34}$$

where

$$\mathcal{L}_f^\varepsilon(\boldsymbol{x},\theta)=\frac{1}{N_f^\varepsilon}\sum_{i=1}^{N_f^\varepsilon}|-\alpha\Delta\varepsilon_i+\beta\nabla\varepsilon_i+\varepsilon_i-\hat{f}_i|^2, \tag{35}$$

and

$$\mathcal{L}_b^\varepsilon(\boldsymbol{x},\theta)=\frac{1}{N_b^\varepsilon}\sum_{i=1}^{N_b^\varepsilon}|\varepsilon_i+\hat{u}_i|^2, \tag{36}$$

and the function $\hat{f}$ is calculated by the formula $\hat{f}_i=\alpha\Delta\hat{u}_i-\beta\nabla\hat{u}_i-\hat{u}_i+f_i$. N_f^ε and N_b^ε represent the dimensionality of the training data satisfying the loss $\mathcal{L}_f^\varepsilon(\boldsymbol{x},\theta)$ and $\mathcal{L}_b^\varepsilon(\boldsymbol{x},\theta)$, respectively. And we balance the discrepancies of each item of the loss function by using the dynamic weight strategy D_1.

In order to achieve the purpose of validation, we consider the model $\varepsilon(\boldsymbol{x},\theta)_L$ in which a 4-layer neural network is used accompanied by 50 neurons per layer. And we train the network for 15,000 gradient descent steps to minimize the loss function by using the Adam optimizer which the initial learning rate is set $\eta=10^{-3}$. Then we consider the L-BFGS-B optimizer to accelerate the convergence of loss function. Besides, we set the other parameters of this study as $N_f^\varepsilon=2500$, $N_b^\varepsilon=800$.

It is natural to ask whether the RPINNs method is efficient to improve numerical accuracy. In order to validate this, we compare the results which are obtained by the multi-grid, PINNs and RPINNs. A comparative study on performance of all methods is summarized in Table 2. When PINNs are used to solve this problem, we adopt a 4-layer neural network with 50 neurons in each layer, and the parameters are set as follows: $N_f=10{,}500$, $N_b=2800$. We train the network for 8000 steps to minimize the loss function by using Adam optimizer, and the L-BFGS-B algorithm is used to accelerate the convergence. It is clear that RPINNs continuously outperform the original PINNs, while RPINNs combined with the dynamic weight strategy (D_1) lead to higher precision prediction, but with higher computational cost. In Fig. 4, more detailed error contours are provided among the multi-grid, PINNs, and RPINNs.

The dynamic weight strategy can effectively mitigate the gradient imbalance pathology and lead to accurate and robust solution. When we adopt the dynamic weight strategy D_0, the results shown in Fig. 5 indicate that $\nabla_\theta\mathcal{L}_b^\varepsilon(\theta_k)$ accumulates at the origin point, while the $\nabla_\theta\mathcal{L}_f^\varepsilon(\theta_k)$ distribute throughout the whole area. This is a clear feature of the imbalanced gradient pathology. In Fig. 6, we show the distribution of back-propagated gradients for $\nabla_\theta\mathcal{L}_b^\varepsilon(\theta_k)$ and $\nabla_\theta\mathcal{L}_f^\varepsilon(\theta_k)$ in which the neural network $\varepsilon(\boldsymbol{x},\theta)_L$ is combined with the strategy D_1. It indicates that RPINNs combined with dynamic weights can alleviate gradient ill-conditions to a certain extent and improve the accuracy of numerical solutions.

To describe the influence of the neural network structure on the numerical solution of RPINNs, we consider a solution obtained by PINNs as a benchmark, and explore the influence by adjusting the number of layers and neurons per layers. The experimental parameters and result are summarized in Table 3. Obviously, as the parameters of the neural network increase, we obtain more accurate numerical solutions.

5.3. Helmholtz equation

In order to show the ability of RPINNs, we consider the RPINNs which is combined with a dynamic weight strategy (D_2) to handle

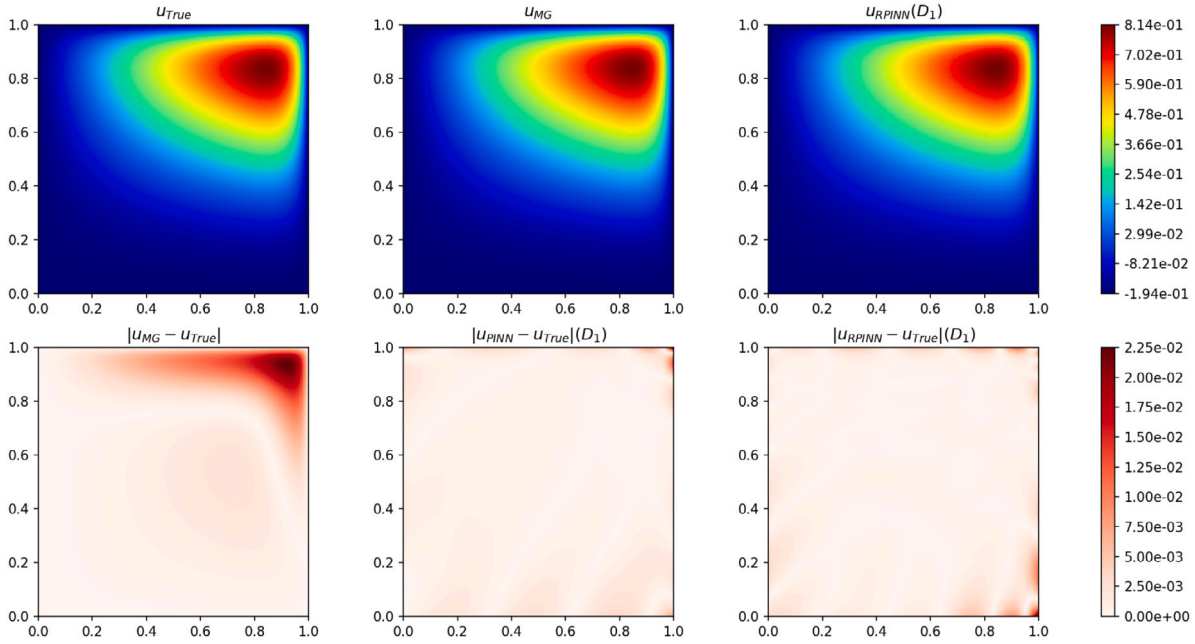


Fig. 4. Comparison of the numerical results between the multi-grid method and the RPINNs, and the absolute error between the PINNs and the RPINNs.

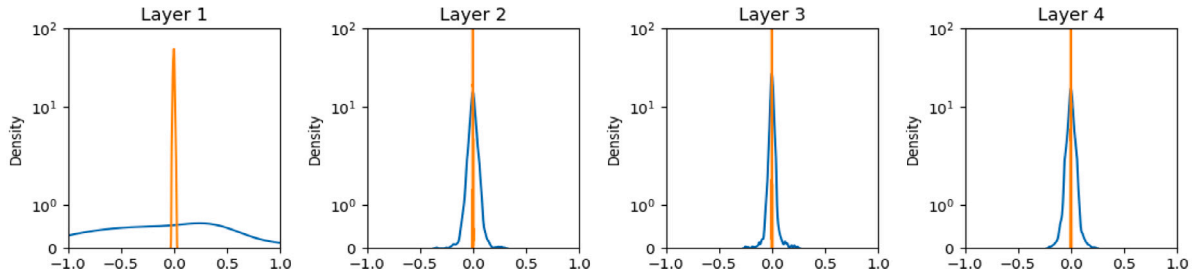


Fig. 5. Histograms of back-propagated gradients for $\nabla_\theta \mathcal{L}_b^\epsilon(\theta_k)$ and $\nabla_\theta \mathcal{L}_f^\epsilon(\theta_k)$ combined with the strategy D_0.

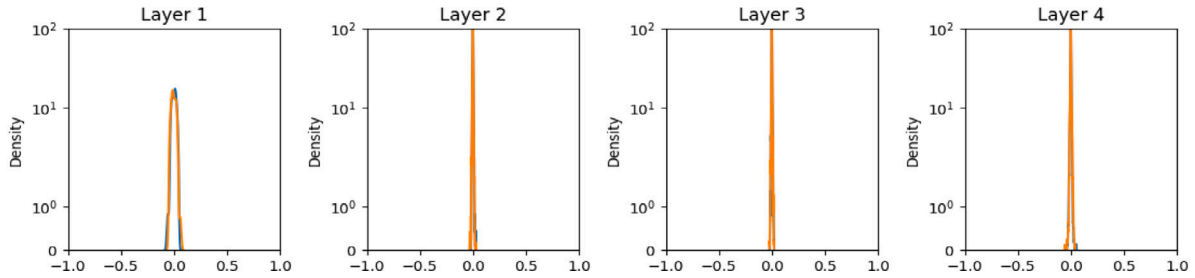


Fig. 6. Histograms of back-propagated gradients for $\nabla_\theta \mathcal{L}_b^\epsilon(\theta_k)$ and $\nabla_\theta \mathcal{L}_f^\epsilon(\theta_k)$ combined with the strategy D_1.

Table 3
Comparison of the influence of neural network structure on the numerical results of RPINNs in the process of correction.

Network structure	Absolute error	Network structure	Absolute error
2 + [20] * 1 + 1	6.2461E−03	2 + [50] * 1 + 1	6.3957E−03
2 + [20] * 2 + 1	6.3840E−03	2 + [50] * 2 + 1	5.2005E−03
2 + [20] * 3 + 1	3.2718E−03	2 + [50] * 3 + 1	9.6843E−04
2 + [20] * 4 + 1	5.7613E−04	2 + [50] * 4 + 1	3.4440E−04
2 + [20] * 5 + 1	1.2793E−04	2 + [50] * 5 + 1	1.0114E−04

high-frequency solutions. Let us consider the Helmholtz equation which is well-known in electromagnetic radiation field [31]. We adopt the equation form as follows:

$$\begin{cases} \Delta u(x,y) + k^2 u(x,y) = q(x,y) & \text{in } \Omega, \\ u(x,y) = h(x,y) & \text{on } \partial\Omega, \end{cases} \tag{37}$$

where $\Omega = [-1,1]^2$. The functions $q(x,y)$ and $h(x,y)$ are considered to be known, and the function $u(x,y)$ denotes the latent function which is represented by neural network. We assume that the problem has a following solution:

$$u(x,y) = \sin(a_1 \pi x)\sin(a_2 \pi y), \tag{38}$$

where we set the parameter $a_1 = 1$ and $a_2 = 4$, and correspondingly, we derive a forcing term $q(x,y)$ which satisfies the equation.

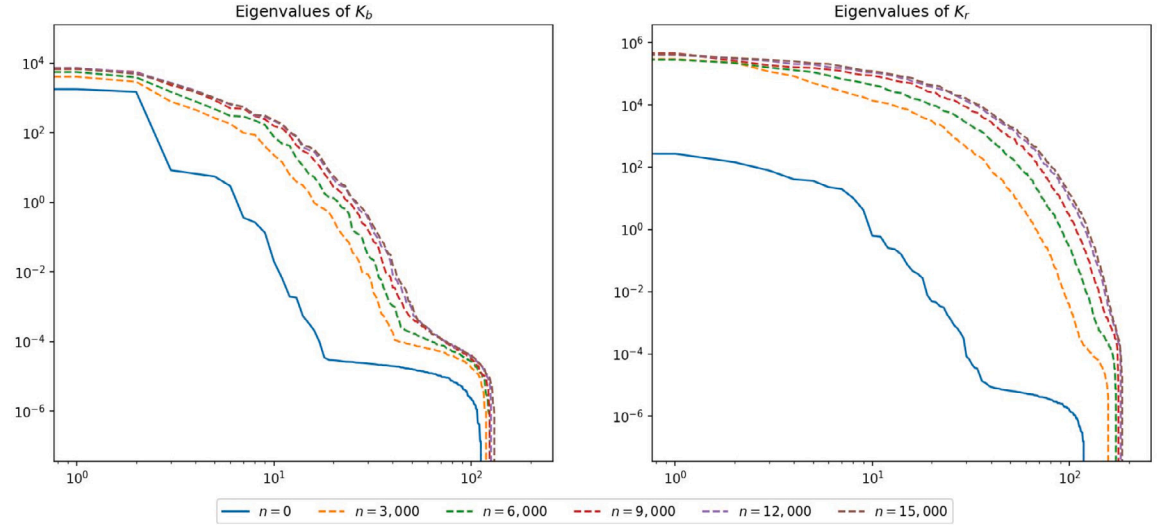


Fig. 7. Two-dimensional Helmholtz equation: Eigenvalues of K_r and K_b at different snapshots during training, stored in descending order.

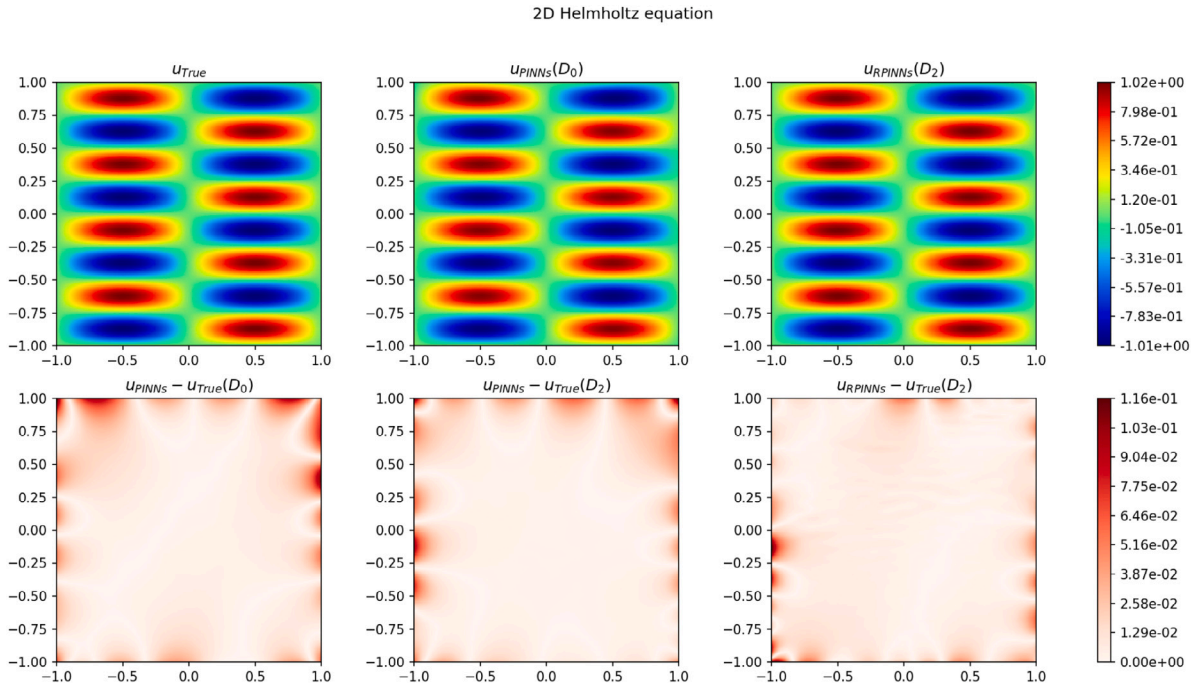


Fig. 8. Comparison of the numerical results between the PINNs and RPINNs, and the absolute error between the PINNs and the RPINNs.

We construct the neural network $f(\boldsymbol{x},\theta)_L$ with 4 hidden layers and 50 neurons per layer, and the loss function is defined as follows:

$$\mathcal{L}(\boldsymbol{x},\theta) = \lambda_f \mathcal{L}_f(\boldsymbol{x},\theta) + \lambda_b \mathcal{L}_b(\boldsymbol{x},\theta), \tag{39}$$

where

$$\mathcal{L}_f(\boldsymbol{x},\theta) = \frac{1}{N_f}\sum_{i=1}^{N_f} |\Delta u_i + k^2 u_i - q_i|^2, \tag{40}$$

and

$$\mathcal{L}_b(\boldsymbol{x},\theta) = \frac{1}{N_b}\sum_{i=1}^{N_b} |u_i - h_i|^2. \tag{41}$$

In the above, the dimensional of training data is $T_f = 8000$, $T_b = 2800$. To quickly obtain a minimum point of the parameters θ, we use full batch gradient decent with an initial learning rate of 10^{-3} and an exponential decay with a decay-rate of 0.9. Meanwhile, we use the dynamic weight strategy D_2 to balance the discrepancies of each item in the loss function. In order to save the computational cost, we define a hyper-parameter variable $K = 200$ to restrict the dimension of NTK matrix K_r, K_b.

To measure the residual information between $\hat{u}$ and u, let us consider to learn the error $\varepsilon = \hat{u} - u$, as well as the latent solution $\epsilon(\boldsymbol{x},\theta)_L$ by training a 4-layer deep neural network. Each of hidden layer contains 40 neurons. Meanwhile, we use the hyper-parameters $N_f^\varepsilon = 2500$, $N_b^\varepsilon = 800$. We train the network for 15,000 gradient descent steps to minimize the loss function by using the Adam optimizer which the initial learning rate is set $\eta = 10^{-3}$. Subsequently, we consider the L-BFGS-B optimizer to accelerate the convergence of loss.

In the dynamic weight strategy (D_2), we have considered the neural tangent kernel of the physics-informed neural network. It can be proved that the NTK of PINNs converges to a deterministic kernel and remains a constant during training. It has been investigated that

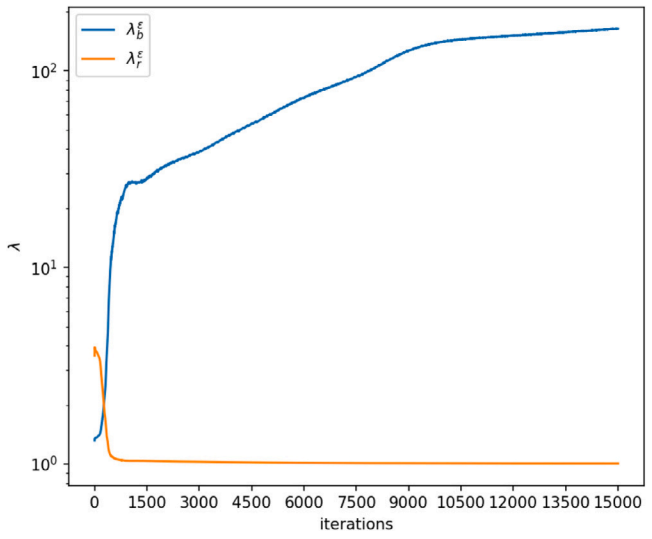


Fig. 9. Two-dimensional Helmholtz equation: the evolution of constants λ_f^ε and λ_b^ε during training.

Table 4
The numerical comparison of PINNs and RPINNs Methods with dynamic weight strategy (D_2) for solving Helmholtz equation.

Method	PINNs (D_0)	PINNs (D_2)	RPINNs (D_0)	RPINNs (D_2)
Time (s)	629.30	764.58	767.58	1075.9
Number of points	13300	13300	13300	13300
Absolute error	8.5356E−03	6.8571E−04	7.4396E−04	6.5973E−04
Relative L_2 error	2.8439E−02	2.3775E−03	2.7909E−03	2.0885E−03

PINNs suffer from the spectral bias. Based on this fact, the learning rate of the component of the objective function corresponding to the kernel eigenvectors with larger eigenvalues is faster. Unfortunately, the eigenvectors corresponding to higher eigenvalues of the NTK matrix have lower frequencies, which lead to the high-frequencies components of the loss function and a lower convergence.

To monitor the change of eigenvalues of the network $\varepsilon(\boldsymbol{x},\theta)_L$, we show the eigenvalues of K_r, K_b for different snapshots during training. A distribution of the eigenvalues of K_r and K_b at the initial stage and the last step of gradient descent are shown in Fig. 7. We infer that the neural tangent kernel does not remain constant. As training iterations increase, the eigenvalues of K_r and K_b is kept almost static, which K_r dominate the K_b during the training. Consequently, $\mathcal{L}_f^\varepsilon(\boldsymbol{x},\theta)$ converges much faster than the loss of boundary conditions.

It is natural to ask whether the RPINNs method with dynamic weight strategy (D_2) is efficient to improve numerical accuracy. We compare the results which obtained by PINNs and RPINNs. A comparative study on the performance of all methods is summarized in Table 4. It is observed that RPINNs combined the dynamic weight strategy (D_2) perform better than PINNs while dealing with the high-frequency solution. By using the dynamic weight strategy based the theory of NTK can effectively improve the calculation accuracy by about 20%. The proposed RPINNs method improves the performance of network in which less more computational cost is taken compared with PINNs. Fig. 8 provides a more intuitive visual assessment of the predictive accuracy for PINNs and RPINNs combined the dynamic weight strategy D_2. Finally, we show in Fig. 9 the evolution of the constants λ_f^ε and λ_b^ε which is used to balance the boundary and inter loss function.

5.4. The Navier–Stokes equations

To verify the performance of RPINNs combined with dynamic weight strategy for high-dimensional nonlinear problem. We consider the three-dimensional incompressible Navier–Stokes equations in a non-dimensional form as follow:

$$\begin{cases} -\dfrac{1}{Re}\Delta\boldsymbol{u} + (\boldsymbol{u}\cdot\nabla)\boldsymbol{u} + \nabla p = \boldsymbol{f} & \text{in } \Omega, \\ \nabla\cdot\boldsymbol{u} = 0 & \text{in } \Omega, \\ \boldsymbol{u} = \boldsymbol{g} & \text{on } \Gamma_D, \end{cases} \tag{42}$$

where $\boldsymbol{u}(x,y,z) = (u(x,y,z), v(x,y,z), w(x,y,z))$ is the velocity vector field, and p is the pressure field and $\Omega = [-1,1]^3$. Re denotes the Reynolds number of the flow. Γ_D is the dirichlet boundary of the flow. To map spatial coordinates (x,y,z) to the latent velocity functions $u(x,y,z)$, $v(x,y,z)$, $w(x,y,z)$ and the latent pressure function $p(x,y,z)$, we consider a fully-connect neural network with 3 input neurons and 4 output neurons, *i.e.*,

$$[x,y,z] \longrightarrow [u(x,y,z), v(x,y,z), w(x,y,z), p(x,y,z)], \tag{43}$$

To assess the accuracy of our models we shall use a solution as follows:

$$\begin{cases} u(x,y,z) = -a[e^{ax}\sin(ay+dz) + e^{az}\cos(ax+dy)], \\ v(x,y,z) = -a[e^{ay}\sin(az+dx) + e^{ax}\cos(ay+dz)], \\ w(x,y,z) = -a[e^{az}\sin(ax+dy) + e^{ay}\cos(az+dx)], \\ p(x,y,z) = -\dfrac{1}{2}a^2[e^{2ax} + e^{2ay} + e^{2az} + 2\sin(ax+dy)\cos(az+dx)e^{a(y+z)} \\ \qquad + 2\sin(ay+dz)\cos(ax+dy)e^{a(z+x)} \\ \qquad + 2\sin(az+dx)\cos(ay+dz)e^{a(x+y)}], \end{cases} \tag{44}$$

where $a = d = 1$ is used. According to the residual of the momentum and continuity equations, we define the loss functions

$$\begin{aligned} \mathcal{L}_f^u &= \frac{1}{N_f}\sum_{i=1}^{N_f} |-\frac{1}{Re}(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}) + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} \\ &\quad + w\frac{\partial u}{\partial z} + \frac{\partial p}{\partial x} - f_1|^2, \\ \mathcal{L}_f^v &= \frac{1}{N_f}\sum_{i=1}^{N_f} |-\frac{1}{Re}(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} + \frac{\partial^2 v}{\partial z^2}) + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} \\ &\quad + w\frac{\partial v}{\partial z} + \frac{\partial p}{\partial y} - f_2|^2, \\ \mathcal{L}_f^w &= \frac{1}{N_f}\sum_{i=1}^{N_f} |-\frac{1}{Re}(\frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2}) + u\frac{\partial w}{\partial x} + v\frac{\partial w}{\partial y} \\ &\quad + w\frac{\partial w}{\partial z} + \frac{\partial p}{\partial z} - f_3|^2, \\ \mathcal{L}_f^c &= \frac{1}{N_f}\sum_{i=1}^{N_f} |\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z}|^2, \end{aligned} \tag{45}$$

In accordance with the boundary conditions, we can formulate the loss functions as follow:

$$\begin{aligned} \mathcal{L}_b^u &= \frac{1}{N_b}\sum_{i=1}^{N_b} |u - g_1|^2, \\ \mathcal{L}_b^v &= \frac{1}{N_b}\sum_{i=1}^{N_b} |v - g_2|^2, \\ \mathcal{L}_b^w &= \frac{1}{N_b}\sum_{i=1}^{N_b} |w - g_3|^2. \end{aligned} \tag{46}$$

We summarize that the loss functions $\mathcal{L}_f(\boldsymbol{x},\theta) = \mathcal{L}_f^u + \mathcal{L}_f^v + \mathcal{L}_f^w + \mathcal{L}_f^c$ and $\mathcal{L}_b(\boldsymbol{x},\theta) = \mathcal{L}_b^u + \mathcal{L}_b^v + \mathcal{L}_b^w$. The total loss function is

$$\mathcal{L}(\boldsymbol{x},\theta) = \lambda_f\mathcal{L}_f(\boldsymbol{x},\theta) + \lambda_b\mathcal{L}_b(\boldsymbol{x},\theta), \tag{47}$$

We choose to represent the latent function $f(\boldsymbol{x},\theta)_L = [u(x,y,z), v(x,y,z), w(x,y,z), p(x,y,z)]$ using a 5-layer neural network with 50

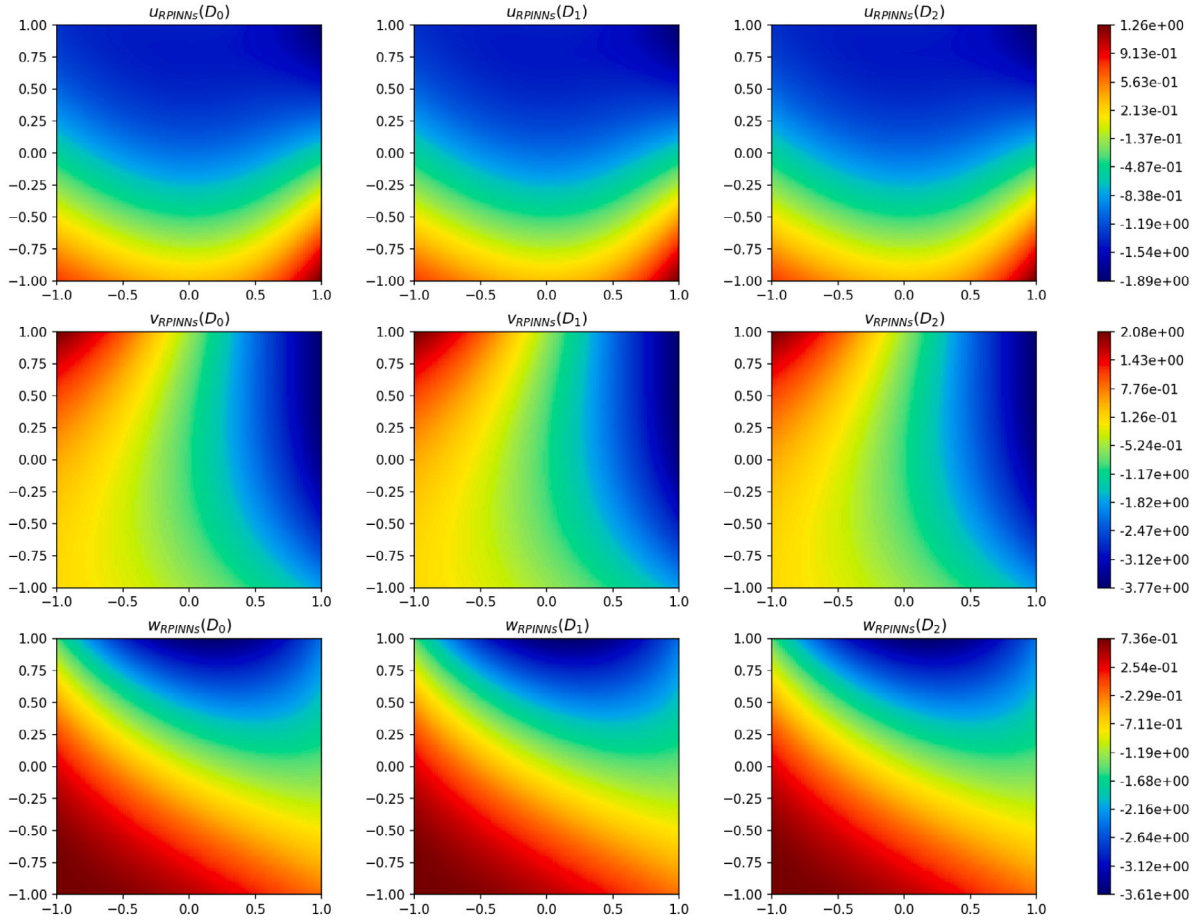


Fig. 10. The comparison of the numerical solution obtained by RPINNs with dynamic weight strategy D_0, D_1 and D_2 on the plane $z = 0$.

Table 5
The comparison of the numerical solution obtained by RPINNs with dynamic weight strategy D_0, D_1 and D_2.

Methods	Training time (s)	Points	Velocity	Absolute error	Relative L_2 error
RPINNs (D_0)	3344.3	15575	u	7.9485E−04	8.1267E−04
			v	6.5121E−04	7.0730E−04
			w	6.4260E−04	6.7787E−04
RPINNs (D_1)	3467.6	15575	u	5.2961E−04	6.1072E−04
			v	6.1580E−04	6.2284E−04
			w	5.6091E−04	5.6201E−04
RPINNs (D_2)	4473.1	15575	u	4.6538E−04	4.9787E−04
			v	5.0516E−04	5.6201E−04
			w	5.1565E−04	5.5728E−04

neurons per layer combined with the hyperbolic tangent function. We set the hyper-parameters $N_f = 8000$, $N_b = 3000$. We obtain the latent solution after 5000 stochastic gradient descent updates using the Adam optimizer. We construct $\epsilon(\boldsymbol{x}, \theta)_L = [\epsilon_u(x, y, z), \epsilon_v(x, y, z), \epsilon_w(x, y, z), \epsilon_p(x, y, z)]$ using the neural network with 40 neurons per layer combined with the hyperbolic tangent function. At the same time, we use the hyper-parameters $N_f^\epsilon = 3375$, $N_b^\epsilon = 1200$. We train the network for 10,000 gradient descent steps to minimize the loss function by using the Adam optimizer which the initial learning rate is set $\eta = 10^{-3}$. Subsequently, we consider the L-BFGS-B optimizer to accelerate the convergence of network.

Fig. 10 provides comparison of the numerical solution obtained by RPINNs with dynamic weight strategies. In Table 5, the results of the experiment are summarized, which show the numerical solution of RPINNs combined the dynamic weight strategy D_0, D_1 and D_2 on the plane $z = 0$. It is observed that RPINNs combined the dynamic weight strategy D_1 and D_2 perform better than RPINNs (D_0) in improving accuracy of numerical solutions.

In Fig. 11, we show the assessment of the evolution of constants λ_f^ϵ and λ_b^ϵ with the dynamic weight strategy D_1 and D_2. It can be observed that the value of λ_b^ϵ with the strategy D_1 is oscillating. The range of λ_f^ϵ and λ_b^ϵ for the two dynamic weight strategies falls within the same interval.

6. Conclusions

In this study, based on the idea of multigrid, we propose the rectified physics-informed neural network for stationary partial differential equations. Specifically, we compared the discrepancy between automatic differentiation and finite difference to measure the gradient of neural network with respect to input vector. In solving stationary partial differential equations by RPINNs, we transfer numerical solution

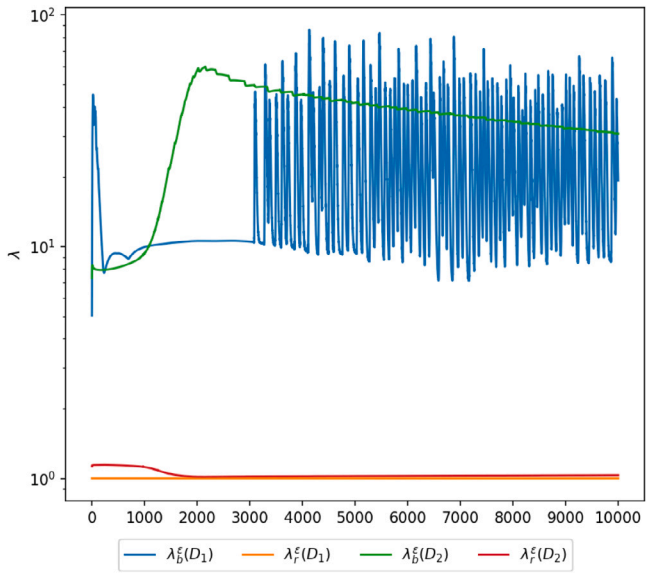


Fig. 11. Evolution diagram of weight coefficients under different weight strategies.

and gradient information that obtained by PINNs to a new fully-connected neural network. A series of numerical investigations show that the proposed RPINNs strategy can improve calculation accuracy of numerical solutions. At the same time, to alleviate gradient ill-conditions, we adopt the dynamic weight strategy. The experimental results show the strategy D_1 based on the gradient information and the strategy D_2 based on the theory of NTK can alleviate discrepancy between various items in the loss function to a certain extent. And we conclude that the weight coefficient obtained by the strategy D_2 is more stable and robust than the strategy D_1. RPINNs combined with dynamic weight strategy can effectively improve calculation accuracy of numerical solutions when calculating stationary partial differential equations. Although this work can improve the calculation accuracy of neural network solving equations, how to quickly obtain high-precision numerical solutions is still a subject which remains to be solved. Meanwhile, a rigorous theoretical proof of RPINNs is still not given, which is an ongoing research.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Acknowledgements

This work is supported by the National Natural Science Foundation of China (Nos. 92152301 and 91852106) and the Autonomous Region Key Laboratory Open Project (2020D04002)

References

[1] Krizhevsky A, Sutskever I, Hinton GE. Imagenet classification with deep convolutional neural networks. Adv Neural Inf Process Syst 2012;25.
[2] Baldi P, Sadowski P, Whiteson D. Searching for exotic particles in high-energy physics with deep learning. Nature Commun 2014;5(1):1–9.
[3] LeCun Y, Bengio Y. Convolutional networks for images, speech, and time series. In: The handbook of brain theory and neural networks, vol. 3361, (10). 1995.
[4] Kondor R, Son HT, Pan H, et al. Covariant compositional networks for learning graphs. 2018, arXiv preprint arXiv:1801.02144.
[5] Krizhevsky A, Sutskever I, Hinton GE. Imagenet classification with deep convolutional neural networks. Adv Neural Inf Process Syst 2012;25.
[6] Silver D, Schrittwieser J, Simonyan K, et al. Mastering the game of go without human knowledge. Nature 2017;550(7676):354–9.
[7] Wang S, Teng Y, Perdikaris P. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. SIAM J Sci Comput 2021;43(5):A3055–81.
[8] Darbon J, Langlois GP, Meng T. Overcoming the curse of dimensionality for some Hamilton–Jacobi partial differential equations via neural network architectures. Res Math Sci 2020;7(3):1–50.
[9] Darbon J, Meng T. On some neural network architectures that can represent viscosity solutions of certain high dimensional Hamilton–Jacobi partial differential equations. J Comput Phys 2021;425:109907.
[10] Raissi M, Perdikaris P, Karniadakis GE. Physics informed deep learning (Part I): Data-driven solutions of nonlinear partial differential equations. 2017, arXiv preprint arXiv:1711.10561.
[11] Raissi M, Perdikaris P, Karniadakis GE. Physics informed deep learning (part II): Data-driven solutions of nonlinear partial differential equations. 2017, arXiv preprint arXiv:1711.10561.
[12] Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J Comput Phys 2019;378:686–707.
[13] Lu L, Meng X, Mao Z, et al. DeepXDE: A deep learning library for solving differential equations. SIAM Rev 2021;63(1):208–28.
[14] Jin X, Cai S, Li H, et al. NSFnets (Navier–Stokes flow nets): Physics-informed neural networks for the incompressible Navier–Stokes equations. J Comput Phys 2021;426:109951.
[15] Wessels H, enfels CWeiß , Wriggers P. The neural particle method–An updated Lagrangian physics informed neural network for computational fluid dynamics. Comput Methods Appl Mech Engrg 2020;368:113127.
[16] Kissas G, Yang Y, Hwuang E, et al. Machine learning in cardiovascular flows modeling: Predicting arterial blood pressure from non-invasive 4D flow MRI data using physics-informed neural networks. Comput Methods Appl Mech Eng 2020;358:112623.
[17] Dwivedi V, Parashar N, Srinivasan B. Distributed physics informed neural network for data-efficient solution to partial differential equations. 2019, arXiv preprint arXiv:1907.08967.
[18] Kharazmi E, Zhang Z, Karniadakis GE. Variational physics-informed neural networks for solving partial differential equations. 2019, arXiv preprint arXiv:1912.00873.
[19] Mishra S, Molinaro R. Estimates on the generalization error of physics-informed neural networks for approximating PDEs. IMA J Numer Anal 2022.
[20] De Ryck T, Mishra S. Error analysis for physics informed neural networks (PINNs) approximating Kolmogorov PDEs. 2021, arXiv preprint arXiv:2106.14473.
[21] Shin Y, Darbon J, Karniadakis GE. On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type PDEs. 2020, arXiv preprint arXiv:2004.01806.
[22] Lanthaler S, Mishra S, Karniadakis GE. Error estimates for deeponets: A deep learning framework in infinite dimensions. Trans Math Appl 2022;6(1):tnac001.
[23] Jacot A, Gabriel F, Hongler C. Neural tangent kernel: Convergence and generalization in neural networks. Adv Neural Inf Process Syst 2018;31.
[24] Margossian CC. A review of automatic differentiation and its efficient implementation. Wiley Interdiscip Rev Data Min Knowl Discov 2019;9(4):1305.
[25] Stein M. Large sample properties of simulations using Latin hypercube sampling. Technometrics 1987;29(2):143–51.
[26] Kingma DP, Ba J. Adam: A method for stochastic optimization. 2014, arXiv preprint arXiv:1412.6980.
[27] Byrd RH, Lu P, Nocedal J, et al. A limited memory algorithm for bound constrained optimization. SIAM J Sci Comput 1995;16(5):1190–208.
[28] Glorot X, Bengio Y. Understanding the difficulty of training deep feedforward neural networks. In: Proceedings of the thirteenth international conference on artificial intelligence and statistics. JMLR Workshop and Conference Proceedings. 2010. p. 249–56.
[29] Wang S, Yu X, Perdikaris P. When and why PINNs fail to train: A neural tangent kernel perspective. J Comput Phys 2022;449:110768.
[30] Ronen B, Jacobs D, Kasten Y, et al. The convergence rate of neural networks for learned functions of different frequencies. Adv Neural Inf Process Syst 2019;32.
[31] Nédélec JC. Acoustic and electromagnetic equations: Integral representations for harmonic problems. Springer Science & Business Media; 2001.